Callbreak Game Monte Carlo Simulation

Vedant Arora , 29th June 2026

University of Delhi

Objective

The goal of this project is to model and predict player performance in the card game Callbreak using a **Monte Carlo Simulation approach**.Instead of relying on short-term static averages, I used historical game data from 14 real games to simulate thousands of hypothetical rounds. This allows us to observe how random luck naturally neutralizes over time, revealing the true underlying efficiency of each player's playstyle.

The Dataset

We track 4 players with highly distinct playstyles:

- **Player P & V:** Conservative, low-risk players who focus on consistent, smaller wins.
- **Player M & U:** Aggressive, high-risk players who look for massive point hauls but carry a higher probability of getting their bids broken.

In [9]:
```
import statistics as st
import random
import matplotlib.pyplot as plt
p = [3.2,1,3.2,2.1,-1,3.1,2,2,2.1,3.2,1.3,2.1,1.1,3]
m = [1.1,-3,4.1,4.1,5.1,-1,4,3,1.1,1,4,5.1,2.2,4]
u = [4.1,4.1,1,3.1,4.1,3.2,4.2,1.2,5.2,-5,1.1,2.1,4,-4]
v = [1,2.3,1.1,1,1.1,2.2,1,3.2,-2,3.1,2.1,1,1.3,2]
```

In [10]:
```
def avg_increment(player):
    s = 0
    for i in player:
        if i>0:
            s+=i
    return s/len(player)
def avg_decrement(player):
    global c
    n = 0
    c = 0
    for i in player:
        if i<0:
```

```
            n-=i
            c+=1
    if c==0:
        return 0,0
    return n/c,c
```

The Continuation Engine

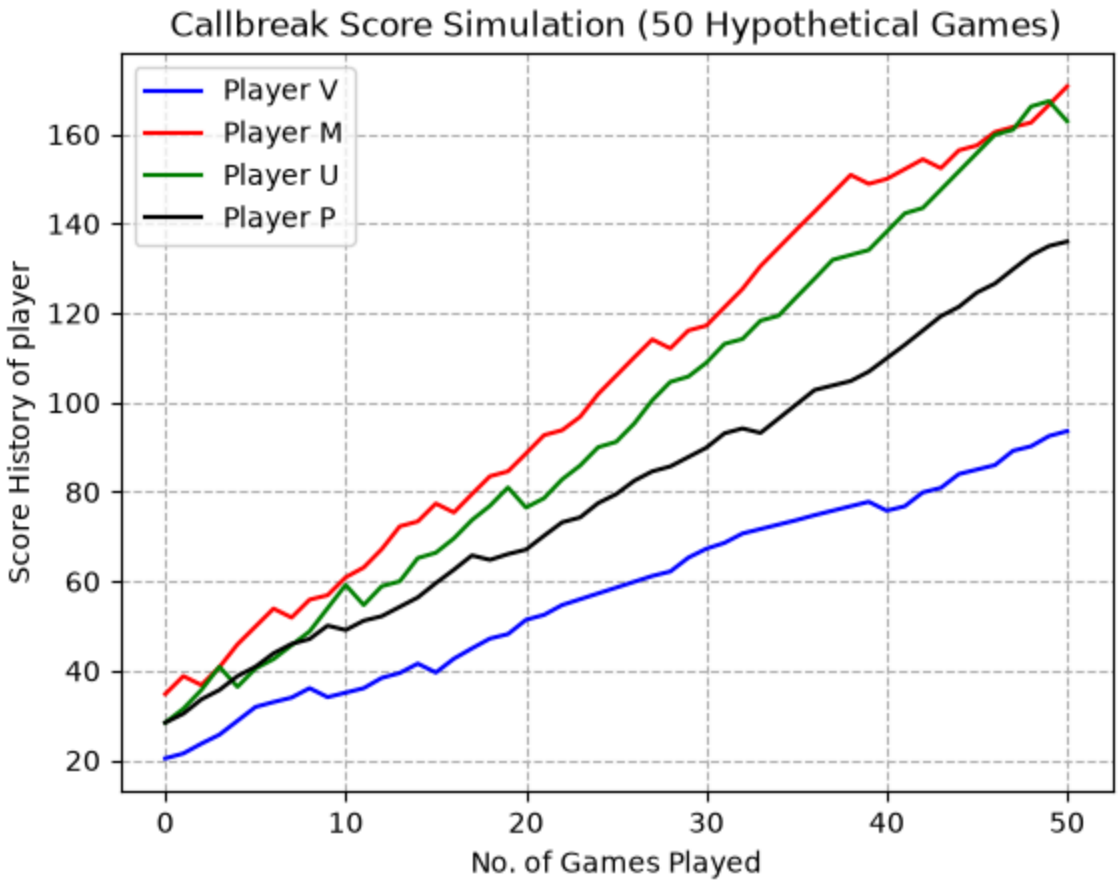
The `cont(player, m)` function acts as our simulation engine. It calculates a player's true historical probability of getting broken (`loss_chance`). For every simulated game, it runs a random trial:

- If the random roll falls within the `loss_chance`, the player suffers their historical average penalty.
- If they win, it randomly samples an actual positive scoring round from their past data.

If We Played 50 More Games

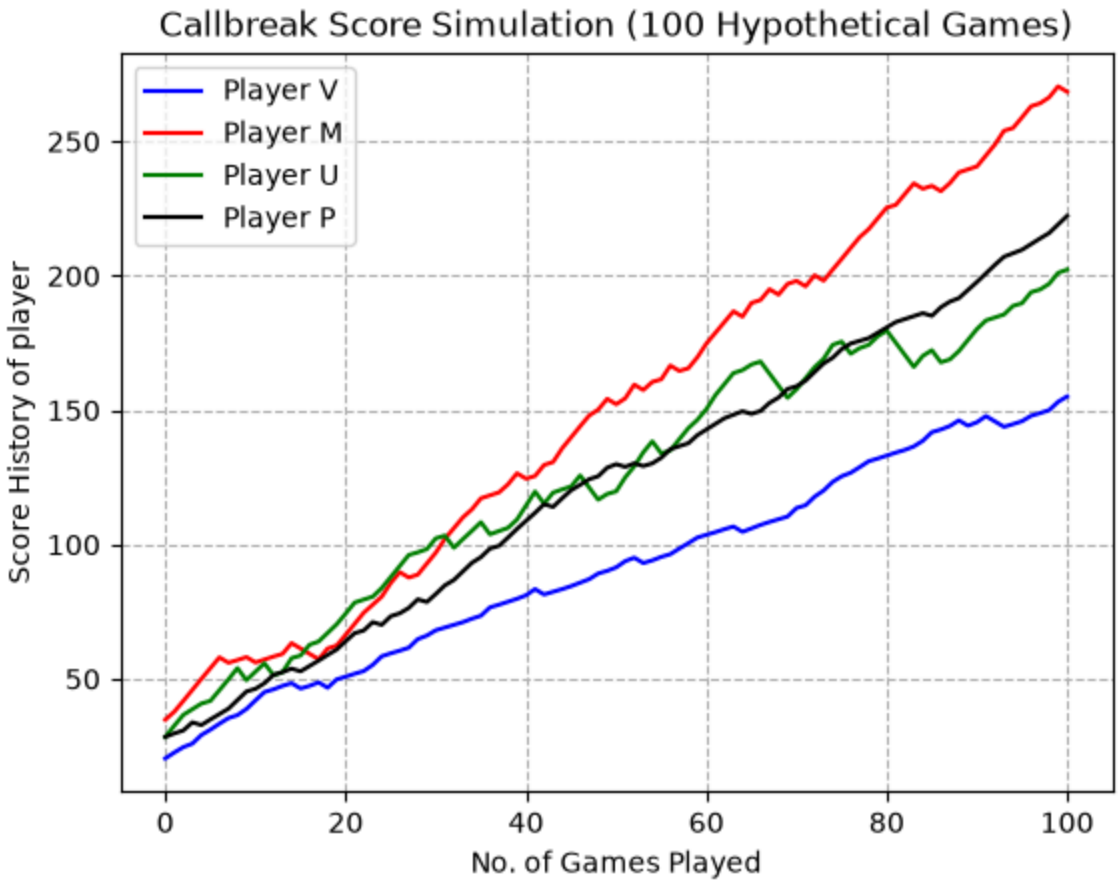
In [11]:
```
def cont(player,m):
    dec_value, c = avg_decrement(player)
    inc_value = avg_increment(player)
    loss_chance = c/14
    score = sum(player)
    score_history = [score]
    for i in range(m):
        if loss_chance>random.random():
            score -= dec_value
        else:
            inc = -1
            while inc<0:
                inc = random.choice(player)
            score+=inc
        score_history.append(score)
    return score_history

plt.plot(cont(v,50),label=f"Player V",color="blue")
plt.plot(cont(m,50),label=f"Player M",color="red")
plt.plot(cont(u,50),label=f"Player U",color="green")
plt.plot(cont(p,50),label=f"Player P",color="black")
plt.title("Callbreak Score Simulation (50 Hypothetical Games)")
plt.xlabel("No. of Games Played")
plt.ylabel("Score History of player")
plt.legend()
plt.grid(True, linestyle="--")
plt.show()
```



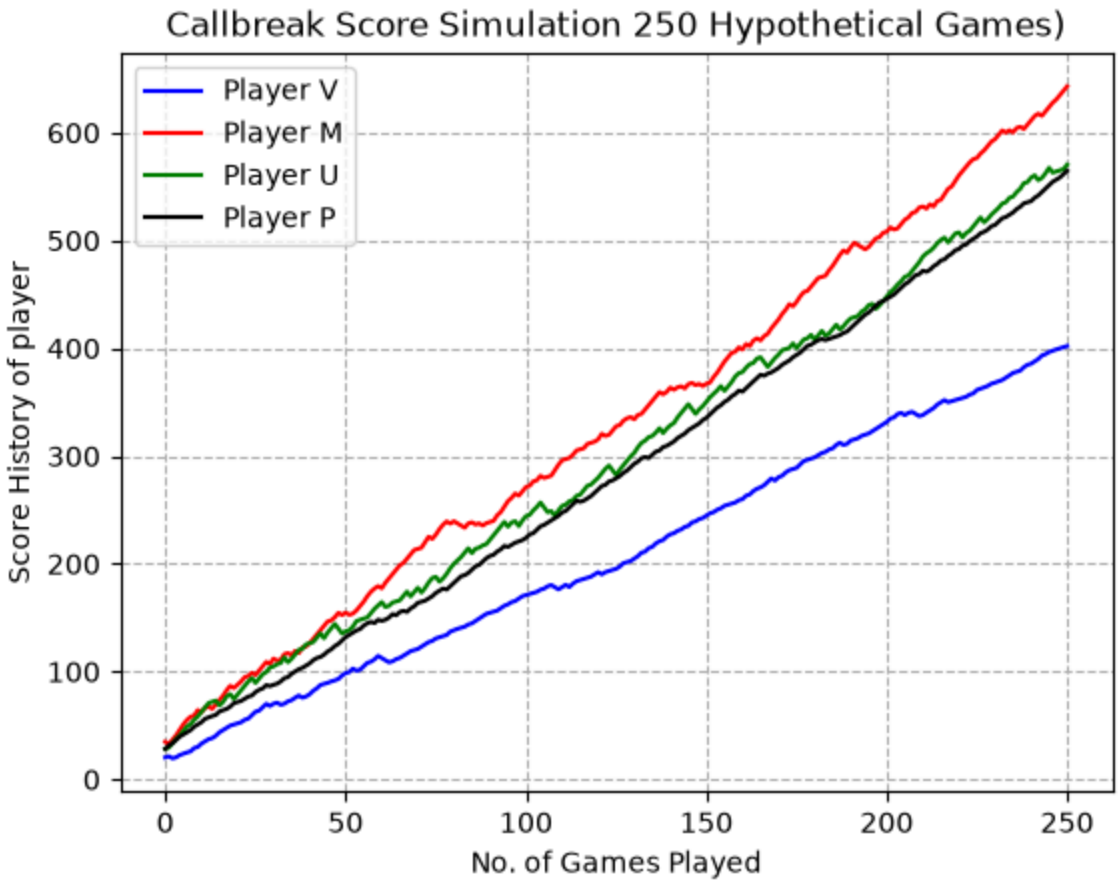
If We Played 100 More Games

In [12]:
```
plt.plot(cont(v,100),label=f"Player V",color="blue")
plt.plot(cont(m,100),label=f"Player M",color="red")
plt.plot(cont(u,100),label=f"Player U",color="green")
plt.plot(cont(p,100),label=f"Player P",color="black")
plt.title("Callbreak Score Simulation (100 Hypothetical Games)")
plt.xlabel("No. of Games Played")
plt.ylabel("Score History of player")
plt.legend()
plt.grid(True, linestyle="--")
plt.show()
```



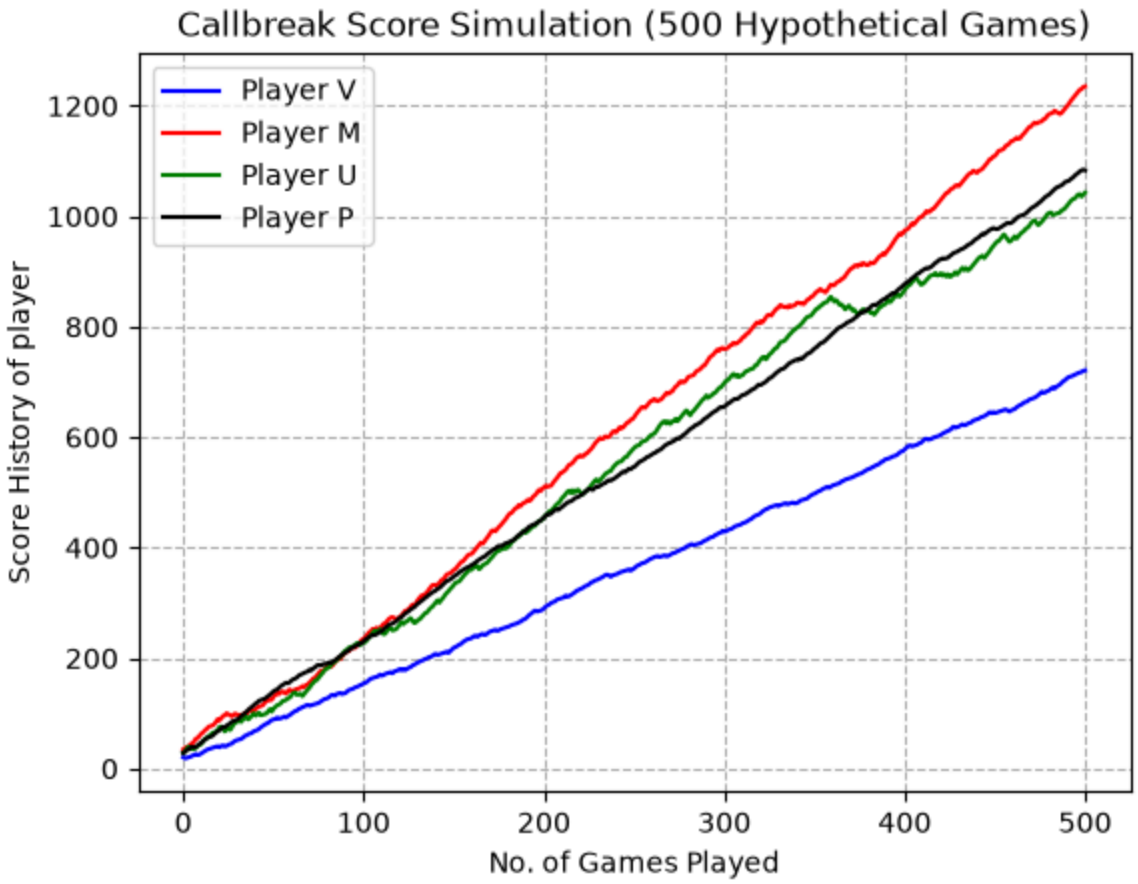
If We Played 250 More Games

In [13]:
```
plt.plot(cont(v,250),label=f"Player V",color="blue")
plt.plot(cont(m,250),label=f"Player M",color="red")
plt.plot(cont(u,250),label=f"Player U",color="green")
plt.plot(cont(p,250),label=f"Player P",color="black")
plt.title("Callbreak Score Simulation 250 Hypothetical Games)")
plt.xlabel("No. of Games Played")
plt.ylabel("Score History of player")
plt.legend()
plt.grid(True, linestyle="--")
plt.show()
```



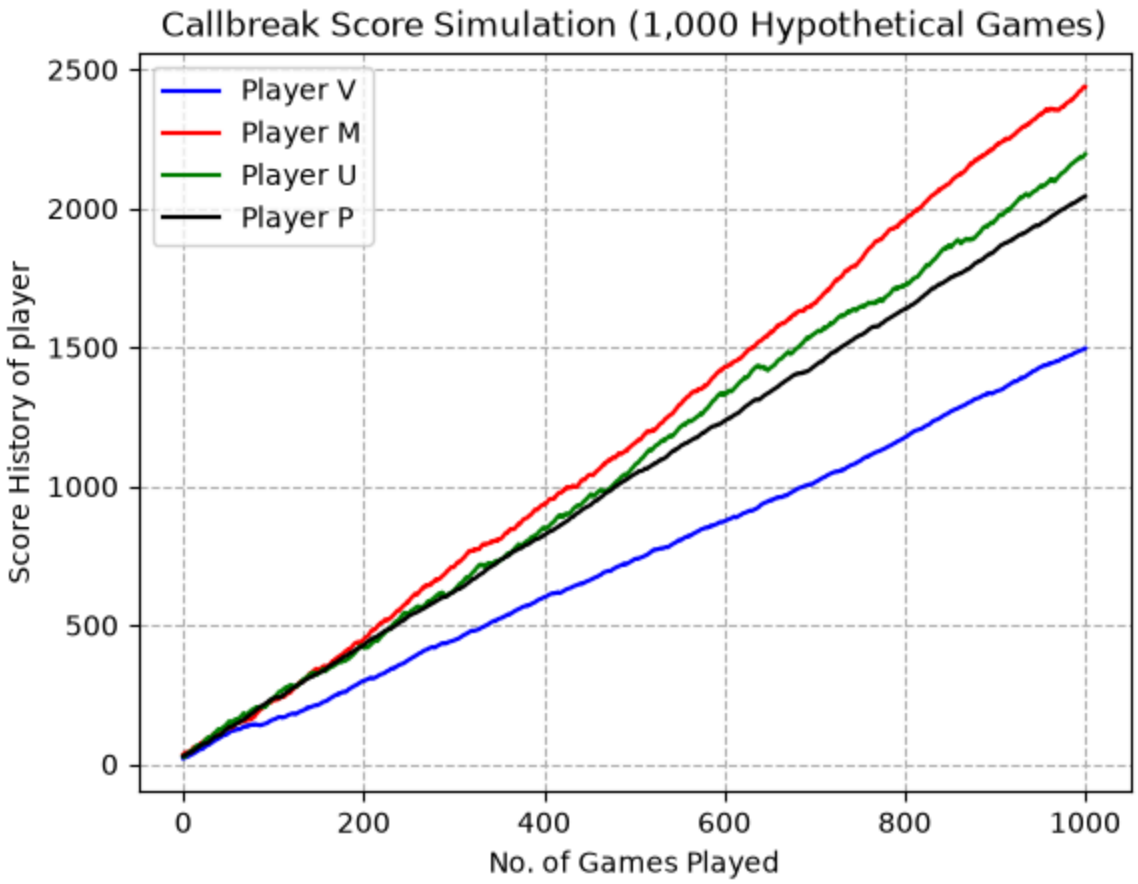
If We Played 500 More Games

In [14]:
```
plt.plot(cont(v,500),label=f"Player V",color="blue")
plt.plot(cont(m,500),label=f"Player M",color="red")
plt.plot(cont(u,500),label=f"Player U",color="green")
plt.plot(cont(p,500),label=f"Player P",color="black")
plt.title("Callbreak Score Simulation (500 Hypothetical Games)")
plt.xlabel("No. of Games Played")
plt.ylabel("Score History of player")
plt.legend()
plt.grid(True, linestyle="--")
plt.show()
```



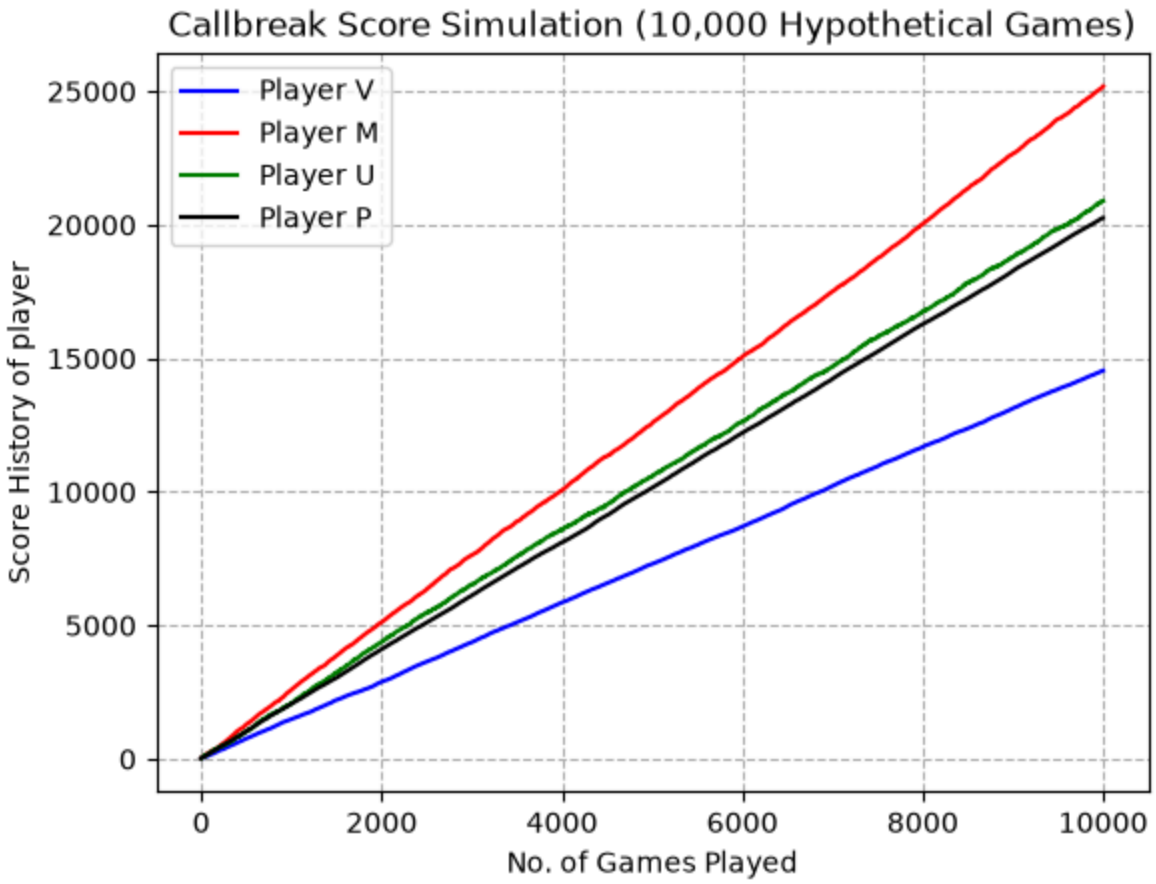
If We Played 1000 More Games

In [15]:
```
plt.plot(cont(v,1000),label=f"Player V",color="blue")
plt.plot(cont(m,1000),label=f"Player M",color="red")
plt.plot(cont(u,1000),label=f"Player U",color="green")
plt.plot(cont(p,1000),label=f"Player P",color="black")
plt.title("Callbreak Score Simulation (1,000 Hypothetical Games)")
plt.xlabel("No. of Games Played")
plt.ylabel("Score History of player")
plt.legend()
plt.grid(True, linestyle="--")
plt.show()
```



If We Played 10,000 More Games (Concluding Simulation)

In [16]:
```
plt.plot(cont(v,10000),label=f"Player V",color="blue")
plt.plot(cont(m,10000),label=f"Player M",color="red")
plt.plot(cont(u,10000),label=f"Player U",color="green")
plt.plot(cont(p,10000),label=f"Player P",color="black")
plt.title("Callbreak Score Simulation (10,000 Hypothetical Games)")
plt.xlabel("No. of Games Played")
plt.ylabel("Score History of player")
plt.legend()
plt.grid(True, linestyle="--")
plt.show()
```



📉 The Law of Large Numbers: Removing the Luck Factor

An essential part of this experiment is observing how the scale of the simulation changes the predictability of the game:

1. **Short Term Chaos:** At low iterations, **Luck is the dominant factor**. The short term starting scores create an initial offset. If you change the number of simulations slightly, temporary streaks wildly distort the trends and rankings constantly flip.
2. **Long Term Convergence:** When I scaled up to a massive number of rounds, individual instances of "good luck" and "bad luck" perfectly balance out to zero. The random noise hits 0 and the lines smooth out. While the initial starting score remains a minor hidden constant in the raw numbers.